

Java Backend Architecture

Interview Series

Chapter 9.3

Application-Level Caching

50+ Interview Q&A

Code Examples

Architecture Diagrams

Advanced Topics

Chapter 9.3 – Application-Level Caching

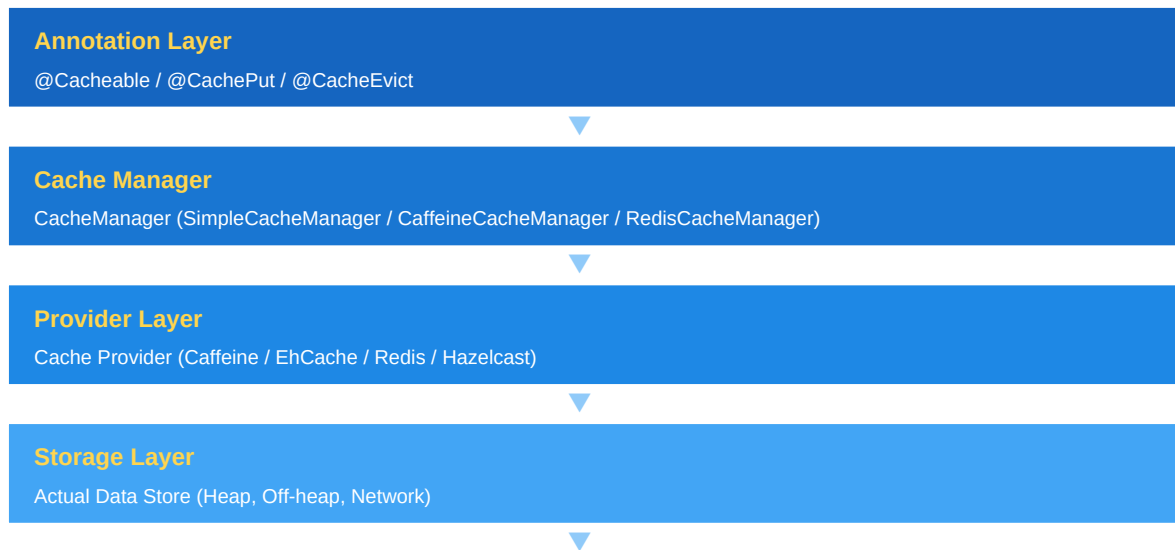
Introduction

Application-level caching sits inside the JVM process — between the business logic and the data store. Unlike CPU caches or JVM heap tricks, **app-level caches are developer-controlled**: you decide what to store, for how long, and how to invalidate. Spring's Cache Abstraction provides a uniform annotation-driven API that decouples business code from the underlying cache provider (Caffeine, EhCache, Redis, Hazelcast). Caffeine is the de-facto in-process choice for Java applications, offering near-optimal hit rates via its W-TinyLFU algorithm.

Core Concepts & Definitions

Cache Abstraction	Spring layer that maps @Cacheable et al. to a CacheManager, decoupling business code from provider.
@Cacheable	Marks a method result to be stored in a named cache; on subsequent calls with same key, method is bypassed.
@CachePut	Always executes the method AND stores the result; useful for updates.
@CacheEvict	Removes one or all entries from a named cache; can be triggered before or after the method.
@Caching	Groups multiple cache annotations on a single method.
CacheManager	Factory that creates and manages named Cache instances; implementation-specific.
Caffeine	High-performance in-process cache for Java implementing W-TinyLFU eviction.
W-TinyLFU	Window + TinyLFU: combines recency (LRU window) and frequency (count-min sketch) for near-optimal hit rates.
Cache Stampede	Dog-pile / thundering herd: many threads miss simultaneously on an expired entry and all fetch from DB.
PER / XFetch	Probabilistic Early Recompute: randomly refresh a cache entry before it expires to avoid stampede.
Refresh-Ahead	Background thread proactively reloads a cache entry before TTL expires.
Key Generator	Strategy to compute the cache key from method parameters; default uses all params.
Condition / Unless	SpEL expressions to conditionally cache or skip storing a result.

Spring Cache Abstraction Architecture



Spring Cache Abstraction – Layered Architecture

The annotation layer (@Cacheable, @CachePut, @CacheEvict) generates AOP proxies that delegate to CacheManager. Swapping from CaffeineCacheManager to RedisCacheManager requires zero business-code changes.

Spring Boot Cache Setup

1. Maven/Gradle dependency:

```

<!-- pom.xml --> <dependency> <groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-cache</artifactId> </dependency> <dependency>
<groupId>com.github.ben-manes.caffeine</groupId> <artifactId>caffeine</artifactId>
<version>3.1.8</version> </dependency>
  
```

2. Enable caching and configure Caffeine:

```

@Configuration @EnableCaching public class CacheConfig { @Bean public CacheManager
cacheManager() { CaffeineCacheManager mgr = new CaffeineCacheManager();
mgr.setCaffeine(Caffeine.newBuilder() .maximumSize(10_000)
.expireAfterWrite(Duration.ofMinutes(10)) .recordStats()); // enable hit/miss metrics return
mgr; } }
  
```

3. Annotate service methods:

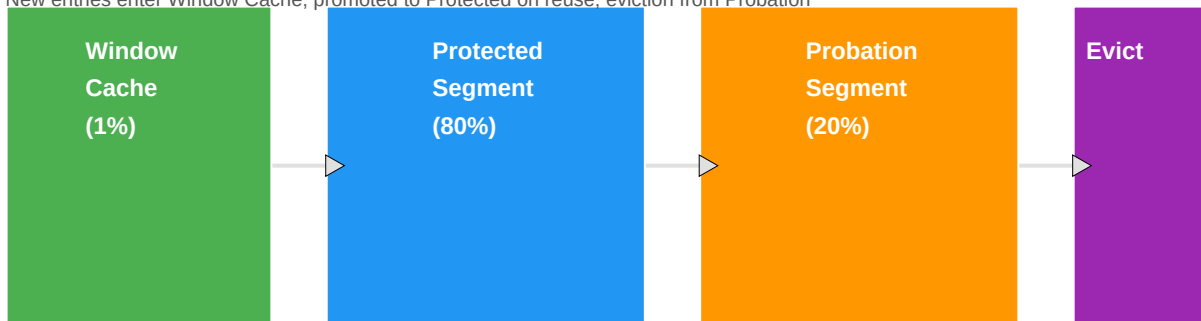
```

@Service public class ProductService { // Cache result; key = productId @Cacheable(value =
"products", key = "#productId") public Product findById(Long productId) { return
productRepo.findById(productId).orElseThrow(); } // Update and refresh cache @CachePut(value =
"products", key = "#product.id") public Product save(Product product) { return
productRepo.save(product); } // Evict single entry @CacheEvict(value = "products", key =
"#productId") public void delete(Long productId) { productRepo.deleteById(productId); } //
Evict entire cache @CacheEvict(value = "products", allEntries = true) public void clearAll() {}
}
  
```

Caffeine & W-TinyLFU Algorithm

W-TinyLFU: Frequency Sketch + Segmented LRU

New entries enter Window Cache; promoted to Protected on reuse; eviction from Probation



Caffeine near-optimal hit rate via admission filter & frequency sketch

Caffeine uses a **Window Cache** (1% capacity) for newly admitted entries, a **Protected Segment** (80%) for frequently accessed entries, and a **Probation Segment** (20%) as a trial area. Eviction compares the frequency (via Count-Min Sketch) of the candidate vs. a victim from the probation segment — the less-frequent entry is evicted. This achieves near-optimal hit rates comparable to OPT.

```
// Programmatic Caffeine usage (without Spring)
Cache<Long, Product> cache =
Caffeine.newBuilder() .maximumSize(5_000) .expireAfterWrite(10, TimeUnit.MINUTES)
.expireAfterAccess(5, TimeUnit.MINUTES) .refreshAfterWrite(8, TimeUnit.MINUTES) // async
refresh .removalListener((k, v, cause) -> log.debug("Evicted key={} cause={}", k, cause))
.recordStats() .build(); // Get-or-load atomically (prevents stampede for single key)
Product p = cache.get(id, productRepo::findById); // Stats
CacheStats stats = cache.stats(); double hitRate = stats.hitRate(); // e.g. 0.94
long evictions = stats.evictionCount();
```

LoadingCache & AsyncLoadingCache

```
// LoadingCache: auto-load on miss
LoadingCache<Long, Product> loadingCache =
Caffeine.newBuilder() .maximumSize(10_000) .refreshAfterWrite(Duration.ofMinutes(5))
.build(productRepo::findById); // CacheLoader
Product p = loadingCache.get(42L); // loads if absent
// AsyncLoadingCache: non-blocking refresh
AsyncLoadingCache<Long, Product> asyncCache =
Caffeine.newBuilder() .maximumSize(10_000) .refreshAfterWrite(Duration.ofMinutes(5))
.buildAsync(key -> CompletableFuture.supplyAsync( () -> productRepo.findById(key)));
CompletableFuture<Product> future = asyncCache.get(42L);
```

Cache Stampede Prevention

WITHOUT Protection

Cache expires → N threads miss simultaneously
 → N DB queries fire at once (stampede)
 → DB overload / timeout

WITH Protection (Lock/PER)

1 thread reloads; others wait on lock
 OR: Probabilistic Early Recompute (PER)
 → Single DB query, cache warmed

Stampede Prevention Techniques

Mutex Lock

Semaphore

PER / XFetch

Background Refresh

Staggered TTL

Dog-pile / thundering-herd: avoid with distributed lock or probabilistic early refresh

```
// 1. Mutex / synchronized approach private final Map<Long, Object> locks = new
ConcurrentHashMap<>(); public Product getWithLock(Long id) { Product cached =
cache.getIfPresent(id); if (cached != null) return cached; Object lock =
locks.computeIfAbsent(id, k -> new Object()); synchronized (lock) { cached =
cache.getIfPresent(id); // double-check if (cached != null) return cached; cached =
productRepo.findById(id); cache.put(id, cached); locks.remove(id); return cached; } } // 2.
Caffeine's built-in stampede protection via LoadingCache.get() // Only ONE thread loads; others
wait for the CompletableFuture Product p = loadingCache.get(id); // thread-safe, single-flight
// 3. Probabilistic Early Recompute (PER / XFetch) // Refresh before expiry with probability
proportional to remaining TTL public Product getWithPER(Long id, Duration ttl) {
CachedEntry<Product> entry = extendedCache.get(id); if (entry == null) return load(id); long
remaining = entry.expiresAt() - System.currentTimeMillis(); double beta = 1.0; // tune
aggressiveness double xfetch = -entry.computeTime() * beta * Math.log(Math.random()); if
(xfetch > remaining) { // proactively refresh CompletableFuture.runAsync(() -> reload(id)); }
return entry.value(); }
```

Conditional Caching with SpEL

```
// Only cache if result is not null and id > 0 @Cacheable( value = "products", key = "#id",
condition = "#id > 0", unless = "#result == null" ) public Product findById(Long id) { ... } //
Custom KeyGenerator @Bean public KeyGenerator myKeyGen() { return (target, method, params) ->
target.getClass().getSimpleName() + ":" + method.getName() + ":" + Arrays.toString(params); }
@Cacheable(value = "reports", keyGenerator = "myKeyGen") public Report
generateReport(ReportRequest req) { ... }
```

Cache Metrics & Monitoring

```
// Micrometer + Actuator integration @Bean public CacheManager cacheManager(MeterRegistry
registry) { CaffeineCacheManager mgr = new CaffeineCacheManager();
mgr.setCaffeine(Caffeine.newBuilder() .maximumSize(10_000)
.expireAfterWrite(Duration.ofMinutes(10)) .recordStats()); // Bind Caffeine stats to
Micrometer new CaffeineCacheMetrics(mgr.getCache("products"), "products",
Collections.emptyList()).bindTo(registry); return mgr; } // Actuator endpoint: GET
/actuator/caches // Prometheus metrics: cache_gets_total{result="hit"}, cache_evictions_total
// Alert: hitRate < 0.85 → investigate key distribution or TTL
```

Multiple Named Caches with Different TTLs

```
@Bean public CacheManager cacheManager() { SimpleCacheManager mgr = new SimpleCacheManager();
mgr.setCaches(List.of( buildCache("products", 10_000, Duration.ofMinutes(30)),
buildCache("sessions", 50_000, Duration.ofMinutes(60)), buildCache("configs", 500,
Duration.ofHours(24)) )); return mgr; } private CaffeineCache buildCache(String name, int size,
Duration ttl) { return new CaffeineCache(name, Caffeine.newBuilder() .maximumSize(size)
.expireAfterWrite(ttl) .recordStats() .build()); }
```

Guava Cache (Legacy Reference)

```
// Guava LoadingCache – use Caffeine instead in new code LoadingCache<Long, Product> guavaCache
= CacheBuilder.newBuilder() .maximumSize(5_000) .expireAfterWrite(10, TimeUnit.MINUTES)
.expireAfterAccess(5, TimeUnit.MINUTES) .removalListener(n -> log.info("Removed: {}",
n.getKey())) .build(new CacheLoader<>() { public Product load(Long key) { return
productRepo.findById(key); } }); // Note: Caffeine is the successor to Guava cache and //
provides significantly better performance via W-TinyLFU
```

Interview Questions & Answers

Q1. What is Spring's Cache Abstraction?

A Spring framework layer that maps cache annotations (@Cacheable, @CachePut, @CacheEvict) to a CacheManager via AOP proxies, fully decoupling business logic from the cache provider.

Q2. How does @Cacheable work internally?

Spring generates a CGLIB/JDK proxy. On first call it computes a key (default: method params), checks the named cache, executes the method if not found, stores the result, and returns it. Subsequent calls with the same key bypass the method entirely.

Q3. What is the difference between @Cacheable and @CachePut?

@Cacheable skips method execution on cache hit. @CachePut always executes the method and always stores the result — useful for update operations that should refresh the cache.

Q4. What is the W-TinyLFU algorithm used by Caffeine?

W-TinyLFU combines a small Window Cache (1%) for new entries, a Protected Segment (80%) for frequent ones, and a Probation Segment (20%). Entry admission is gated by a Count-Min Sketch frequency filter, achieving near-optimal hit rates.

Q5. What is a cache stampede and how do you prevent it?

When a hot cache key expires, all concurrent threads miss simultaneously and each makes a DB call. Prevention: use Caffeine's LoadingCache (single-flight per key), a distributed lock (Redis SETNX), probabilistic early recompute (PER/XFetch), staggered TTLs, or background refresh.

Q6. What is Probabilistic Early Recompute (PER)?

PER (XFetch algorithm) refreshes a cache entry before expiry with a probability proportional to $\text{compute_time} \times \beta \times -\log(\text{rand}())$. As TTL approaches zero, probability of refresh approaches 1, ensuring the entry is refreshed by exactly one thread before expiry.

Q7. How do you configure multiple caches with different TTLs in Spring?

Use SimpleCacheManager with a list of CaffeineCache instances, each built with its own Caffeine spec. Or use application.yml: spring.cache.caffeine.spec per cache name.

Q8. What is the default key generation strategy in Spring Cache?

SimpleKeyGenerator: no params → SimpleKey.EMPTY; one param → param itself; multiple params → new SimpleKey(params). Custom generators implement KeyGenerator. SpEL in key attribute overrides this.

Q9. When would you use @CacheEvict(allEntries=true)?

When a bulk update invalidates many entries (e.g., flush product catalog after import). Selectively evicting millions of keys is impractical; allEntries=true clears the named cache in O(1).

Q10. How does Caffeine's refreshAfterWrite differ from expireAfterWrite?

expireAfterWrite removes the entry after TTL; a thread finding it absent must load synchronously. refreshAfterWrite keeps the stale entry, triggers an async reload, and returns the stale value during refresh — reducing latency spikes.

Q11. What cache statistics does Caffeine expose?

CacheStats: hitCount, missCount, hitRate, missRate, loadSuccessCount, loadFailureCount, totalLoadTime, evictionCount, evictionWeight. Accessed via cache.stats(); integrates with Micrometer.

Q12. What is the difference between expireAfterWrite and expireAfterAccess?

expireAfterWrite: expires TTL since the entry was created/updated. expireAfterAccess: expires TTL since last read or write — hot entries stay indefinitely. Use expireAfterAccess for hot/cold workloads, expireAfterWrite for data freshness guarantees.

Q13. How do you test Spring-cached methods?

Option 1: @SpringBootTest with CacheManager injected — call method twice, verify DB repo called once. Option 2: Mock CacheManager. Option 3: Use cache.asMap() to inspect entries. Clear between tests with @CacheEvict or cacheManager.getCache(name).clear().

Q14. What is the condition vs unless attribute in @Cacheable?

condition is evaluated before the method (uses #param SpEL) — if false, caching is skipped entirely. unless is evaluated after the method (uses #result SpEL) — if true, the result is NOT stored. Example: unless='#result==null' prevents caching null values.

Q15. What is a LoadingCache vs a regular Cache in Caffeine?

A regular Cache.get(key) returns null on miss; caller must handle loading. LoadingCache.get(key) automatically calls the CacheLoader on miss; concurrent requests for the same key are de-duplicated — only one loader executes, others wait for the same CompletableFuture.

Q16. How would you implement a write-through cache in Spring?

Use @CachePut on every save/update method: method always executes, result always stored. Alternatively inject CacheManager and manually put after DB write. Pair with @CacheEvict on delete.

Q17. What are the trade-offs of in-process vs distributed caching?

In-process (Caffeine): nanosecond latency, no serialization, but per-node — inconsistent across cluster. Distributed (Redis): shared truth, cluster-consistent, but millisecond latency + serialization overhead. Common pattern: L1 Caffeine (seconds TTL) + L2 Redis (minutes TTL).

Q18. How do you handle cache key collisions?

Use namespaced keys (cache name as prefix), include type discriminator in key, or use a custom KeyGenerator. Choose keys that uniquely identify the query context including tenant/user scope if multi-tenant.

Q19. What is the impact of storing mutable objects in Caffeine?

Caffeine stores object references — mutations to the returned object mutate the cached entry. Always cache immutable objects or defensive copies. Use Caffeine's `.build(loader)` with copy-on-read to ensure safety.

Q20. How does Spring Cache work with transactions?

By default, `@Cacheable` put happens when the proxy method returns — before transaction commit. If the transaction rolls back, the stale entry remains cached. Use `@Transactional` + `@CachePut` carefully or leverage `TransactionSynchronizationManager` to defer cache put to after commit.

Q21. What is cache warming (pre-loading) and why is it important?

Pre-populating a cache before traffic hits it avoids cold-start latency. Strategies: eager load on startup (`@PostConstruct`), background warm-up job, export/import cache state on restart (Caffeine doesn't persist — use Ehcache disk store or external seeding from Redis).

Q22. How do you implement a two-level (L1/L2) cache in Spring?

Create a `CompositeCacheManager` wrapping `CaffeineCacheManager` (L1) and `RedisCacheManager` (L2). On miss from L1, Spring consults L2. On L2 hit, populate L1. Custom `CacheResolver` can implement lookup order and back-fill logic.

Q23. What are the risks of long TTLs in application cache?

Stale data served to users, security issues if permission changes not reflected, inconsistency in cluster (each node caches independently). Mitigate: event-driven invalidation (DB triggers, Kafka events), shorter TTLs for volatile data, explicit eviction on mutation.

Q24. How do you size a Caffeine cache?

`maximumSize` should reflect available heap minus other allocations. Rule of thumb: `cached objects * average object size < 10-20% heap`. Use `maximumWeight` with a `Weigher` for variable-size objects. Monitor eviction rate: high eviction + high miss rate means cache too small.

Q25. What is cache aside vs inline cache (Spring perspective)?

Cache-aside: application explicitly checks cache before calling DB (`@Cacheable` implements this pattern). Inline / read-through: cache intercepts all DB reads transparently (e.g., Hibernate 2nd-level cache). Spring `@Cacheable` is cache-aside — the method itself is the data loader.

Q26. How does Caffeine handle concurrent cache population?

Caffeine's `AsyncLoadingCache` uses `CompletableFuture` per key. If N threads request the same absent key, only one `CompletableFuture` is created and all N threads attach to it — this is the single-flight / request coalescing pattern, preventing stampede at the per-key level.

Q27. What is weight-based eviction in Caffeine?

Instead of bounding by entry count, you bound by total weight. Assign a `Weigher` that returns each entry's cost (e.g., byte size). Caffeine evicts entries when sum of weights exceeds `maximumWeight`. Useful for caches storing variable-sized objects like JSON responses.

Q28. How do you handle cache invalidation across microservices?

Publish cache invalidation events to a message broker (Kafka, Redis Pub/Sub). Each service subscribes and evicts the relevant entry from its local Caffeine cache. This is the cache invalidation bus pattern — eventual consistency with low coupling.

Q29. What is the null-caching problem and how do you avoid it?

If DB returns null (record not found), `@Cacheable` caches null, preventing repeated DB hits for missing keys — this is actually desirable (negative caching). To disable: `unless='#result==null'`. To limit TTL for nulls, use a Caffeine spec with shorter TTL for null entries.

Q30. What Micrometer metrics are automatically provided for Spring Cache?

With `spring-boot-actuator`: `cache.gets` (tagged hit/miss), `cache.puts`, `cache.evictions`, `cache.size`. With `Caffeine` + `recordStats()`: additionally `cache.load.duration`, `cache.eviction.weight`. Expose via `/actuator/metrics/cache.gets?tag=name:products&tag=result:hit`.

Q31. How does Spring's @EnableCaching work?

`@EnableCaching` imports `CachingConfigurationSelector` which registers `AutoProxyRegistrar` (creates AOP proxy infrastructure) and `ProxyCachingConfiguration` (registers `CacheInterceptor` as an AOP advice). The interceptor intercepts annotated methods and delegates to the configured `CacheManager`.

Q32. What is the difference between CaffeineCache and ConcurrentMapCache?

`ConcurrentMapCache` (Spring default with no explicit provider) is unbounded — no eviction, no TTL, grows indefinitely. `CaffeineCache` is bounded with TTL and eviction. Never use `ConcurrentMapCache` in production without size limits; it is a memory leak waiting to happen.

Q33. How would you handle a cache that needs to be shared across cluster nodes with Caffeine?

`Caffeine` is in-process only — use Redis or Hazelcast for sharing. For L1/L2: each node has its own `Caffeine` cache (L1) backed by Redis (L2). Invalidation events (Redis Keyspace Notifications or Kafka) propagate eviction to all L1 caches. This gives both low latency and cluster consistency.

Q34. What serialization formats are most efficient for cached objects in Redis?

Kryo or FST (fastest, binary, Java-specific), Protobuf (cross-language, compact), MessagePack (compact, JSON-like), Smile (binary JSON). Avoid Java default serialization (slow, verbose). Spring Data Redis uses Jackson JSON by default — suitable but not optimal; switch to Kryo for high-throughput services.

Q35. What is a write-behind (write-back) cache and how is it implemented in Spring?

DB write is deferred: data is written to cache immediately, persisted asynchronously. Spring has no built-in write-behind; implement with `@CachePut` + an async `@Scheduled` batch flusher or a dirty-flag pattern. Risk: data loss if node crashes before flush — use write-ahead log or Redis persistence as safety net.

Q36. How do you implement cache-level circuit breaking?

Wrap cache calls with `Resilience4j CircuitBreaker`: if cache provider (Redis) is down, fallback to DB directly without propagating cache errors. Use fallback on `CacheManager` to catch exceptions. `Caffeine` in-process is effectively always available — circuit breaking applies more to remote caches.

Q37. What is singletonCache antipattern?

Using a global static map as a cache — no eviction, no TTL, thread-safety issues, no metrics. Replace with `Caffeine`. If you need a global constant map that never changes, a static final `ImmutableMap` is acceptable, but it is a lookup table, not a cache.

Q38. What is Ehcache and how does it compare to Caffeine?

`Ehcache` is a mature Java cache supporting disk overflow, clustering via `Terracotta`, `JSR-107` (`JCache`). `Caffeine` is purely in-memory, higher performance (W-TinyLFU vs LRU), simpler API. Choose `Ehcache` when you need disk overflow or `JSR-107` compliance; choose `Caffeine` for maximum throughput.

Q39. How does Spring handle the case where CacheManager is not configured?

Spring 4.1+ auto-detects cache providers on classpath. Without any provider, it falls back to `NoOpCacheManager` (caching is disabled) in Spring Boot if `spring.cache.type=none`, or `ConcurrentMapCacheManager` (unbounded) otherwise. Always explicitly configure a `CacheManager` in production.

Q40. What is the JSR-107 (JCache) API and does Caffeine support it?

JSR-107 is a standard Java caching API (javax.cache). Caffeine provides a JCache adapter via caffeine-jcache dependency. Spring supports JSR-107 with `@CacheResult`, `@CacheRemove` annotations (javax.cache.annotation). Useful for portable code that must work with multiple JCache providers.

Q41. Explain the CacheInterceptor chain in Spring.

CacheInterceptor extends CacheAspectSupport. On method invocation: 1) evaluate `@CacheEvict(beforeInvocation=true)`, 2) check `@Cacheable` — return cached value if hit, 3) invoke actual method, 4) process `@CachePut`, 5) process `@CacheEvict(afterInvocation)`. All operations delegated to CacheManager/Cache.

Q42. How do you implement a caching layer for paginated queries?

Key the cache on (page, size, sort, filters) tuple — serialize to string. Cache the Page result. TTL should be short since data changes. On mutation, evict all paginated cache entries with `allEntries=true` or use a generation counter in the key: `'products:v{version}:page{n}'`.

Q43. What is cache coloring / sharding in application caches?

Partitioning cache entries across multiple sub-caches or shards to reduce lock contention. Caffeine handles concurrency internally via its striped/segmented design. In Redis, sharding is implicit via cluster slot assignment. Application-level sharding is rarely needed with Caffeine.

Q44. How would you diagnose a low cache hit rate?

1) Check `CacheStats.hitRate()` — below 0.8 is concerning. 2) Analyze key distribution — are keys diverse (many unique IDs) vs concentrated? 3) Check cache size vs working set size — if working set > maxSize, hit rate will be low. 4) Check TTL vs access pattern — TTL too short for access frequency. 5) Check for cache busters — frequent evictions from code bugs.

Q45. What are ghost caches / shadow entries in Caffeine?

When an entry is evicted from the main cache, Caffeine keeps a lightweight 'ghost' entry (just the key hash) in a history structure. If the evicted key is accessed soon, the frequency counter is already primed — this prevents re-eviction of recently evicted entries (part of W-TinyLFU).

Q46. How do you prevent cache key space explosion?

Key space explosion: unbounded unique keys fill cache, hit rate drops to 0. Prevent by: normalizing/bucketing parameters (round timestamps to hour), using `@Cacheable` condition to skip non-cacheable inputs, adding `maximumSize` eviction, and monitoring eviction rate.

Q47. What happens when you call a @Cacheable method from within the same class?

Spring's AOP proxy is bypassed for self-invocation — the cache is NOT consulted. Solutions: inject the bean into itself (self-injection), move the cached method to a separate `@Component`, or use AspectJ load-time weaving (LTW) instead of proxy-based AOP.

Q48. Describe an advanced pattern combining Spring Cache with event-driven invalidation.

1) `@CachePut` on save stores in Redis. 2) `@TransactionalEventListener(AFTER_COMMIT)` publishes `ProductUpdatedEvent`. 3) Kafka consumer in other services receives event, calls `@CacheEvict`. This ensures cache is only populated after successful DB commit and cross-service invalidation is decoupled via events, not synchronous RPC.